Mastering Design Patterns in Java

Creational, Structural, and Behavioral Patterns

GoF Design Patterns	Real-World Java Examples	Best Practices
Introduction to the Gang of Four patterns and their significance	Practical implementations you can apply immediately	Use cases and guidelines for enterprise-grade Java development

What Are Design Patterns?

Design patterns are **reusable solutions to common software problems**. They improve code maintainability and scalability, helping developers write clean, flexible, and robust code.

Reusability

Apply proven solutions across different projects and contexts without reinventing the wheel.

Better Architecture

Structure your codebase with clarity and purpose, following industry-recognised standards.

Loose Coupling

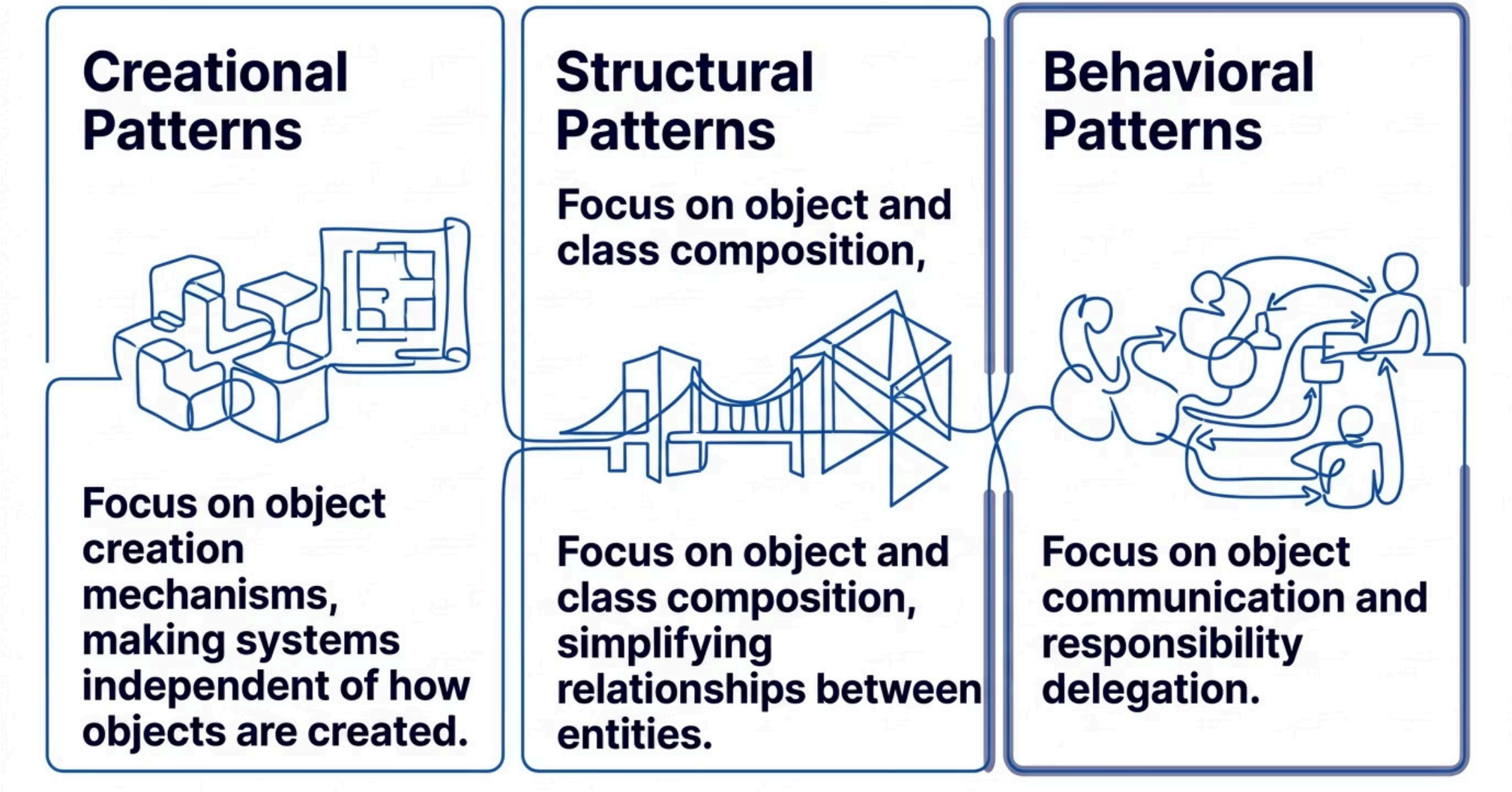
Reduce dependencies between components, making systems easier to modify and extend.

Easier Maintenance

Code that follows design patterns is simpler to debug, update, and hand off to other developers.

Types of GoF Design Patterns

The Gang of Four (GoF) defined **23 design patterns** grouped into three fundamental categories, each addressing a different dimension of software design.



Each category targets a specific layer of design complexity — from how objects are born, to how they relate, to how they communicate.

Creational Patterns Overview

CATEGORY 1 OF 3

Goal

Create objects **efficiently and flexibly**, decoupling the system from the specifics of object instantiation.

Patterns Covered

- Singleton Pattern
- Factory Method Pattern

Common Use Cases

Logging

A single logger instance shared across the entire application.

Database Connections

Controlled creation of connection pool objects.

Object Creation Frameworks

Factories that produce the right object at runtime.

Singleton Pattern

CREATIONAL

Purpose: Ensure only *one instance* of a class exists throughout the entire application lifecycle, providing a global access point to that instance.

Real-World Examples

Logger

One logging service used across all modules.

Configuration Manager

Single source of truth for application settings.

Cache Manager

Centralised cache to avoid duplication.

Key Concepts

Private Constructor

Prevents external instantiation.

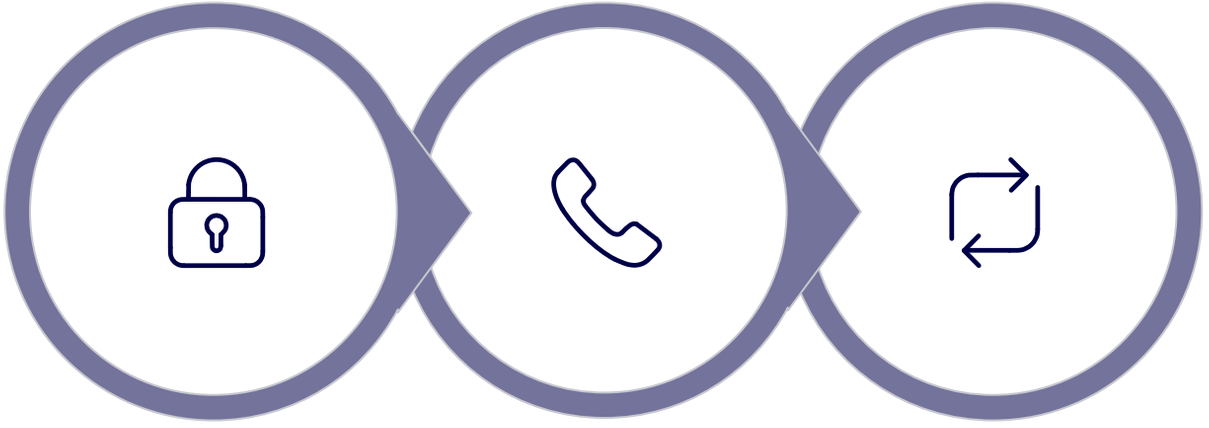
Static Instance

Holds the single object reference.

Global Access Method

`getInstance()` returns the shared object.

Singleton Pattern — In Practice



Private Constructor

Client calls getInstance( )

Same Instance Returned

The Singleton guarantees a single shared instance, making it ideal for resources that are expensive to create or must remain consistent.

1

Thread Safety

Use `synchronized` or double-checked locking to prevent race conditions in multi-threaded environments.

2

Lazy vs Eager Initialisation

Lazy creates the instance on first use; eager creates it at class loading — each has trade-offs.

3

Advantages & Drawbacks

Controlled access is powerful, but overuse can introduce hidden global state and hinder testability.

Factory Method Pattern

CREATIONAL

Purpose: Define an interface for creating objects, but let subclasses decide which class to instantiate — *creating objects without exposing creation logic* to the client.

Real-World Examples

Payment Systems

Create PayPal, Stripe, or bank transfer objects at runtime.

Notification Services

Produce Email, SMS, or Push notification objects dynamically.

Document Generators

Generate PDF, Word, or HTML documents from a common interface.

Benefits

- **Loose coupling** — client code doesn't depend on concrete classes
- **Easier extension** — add new product types without changing existing code
- **Cleaner code** — creation logic is centralised and encapsulated

Factory Method Pattern — In Practice

01

Product Interface

Defines the contract all concrete products must fulfil.

02

Concrete Products

Specific implementations of the product interface (e.g. `PDFDocument`, `WordDocument`).

03

Factory Class

Contains the factory method that decides which product to instantiate.

04

Client Code

Calls the factory method without knowing which concrete class is returned.

The Factory Method Pattern supports the **Open/Closed Principle** — open for extension, closed for modification. New product types can be added without altering existing factory or client code.

Structural Patterns Overview

CATEGORY 2 OF 3

Goal

Simplify **relationships between classes and objects**, making complex structures easier to compose and manage.

Patterns Covered

- Adapter Pattern
- Decorator Pattern

Common Use Cases

Third-Party Integrations

Bridge incompatible APIs and legacy systems with modern code.

UI Frameworks

Compose complex UI components from simpler building blocks.

Java I/O Streams

Layer functionality using decorators like `BufferedReader`.

Adapter Pattern

STRUCTURAL

Purpose: Allow *incompatible interfaces to work together* by wrapping an existing class with a new interface — acting as a translator between two systems.

Real-World Examples

Mobile Charger Adapters

A physical analogy — converting one plug type to another without changing the device.

Legacy System Integration

Wrap old APIs so modern code can consume them seamlessly.

External APIs

Adapt third-party service responses to match your internal data models.

Benefits

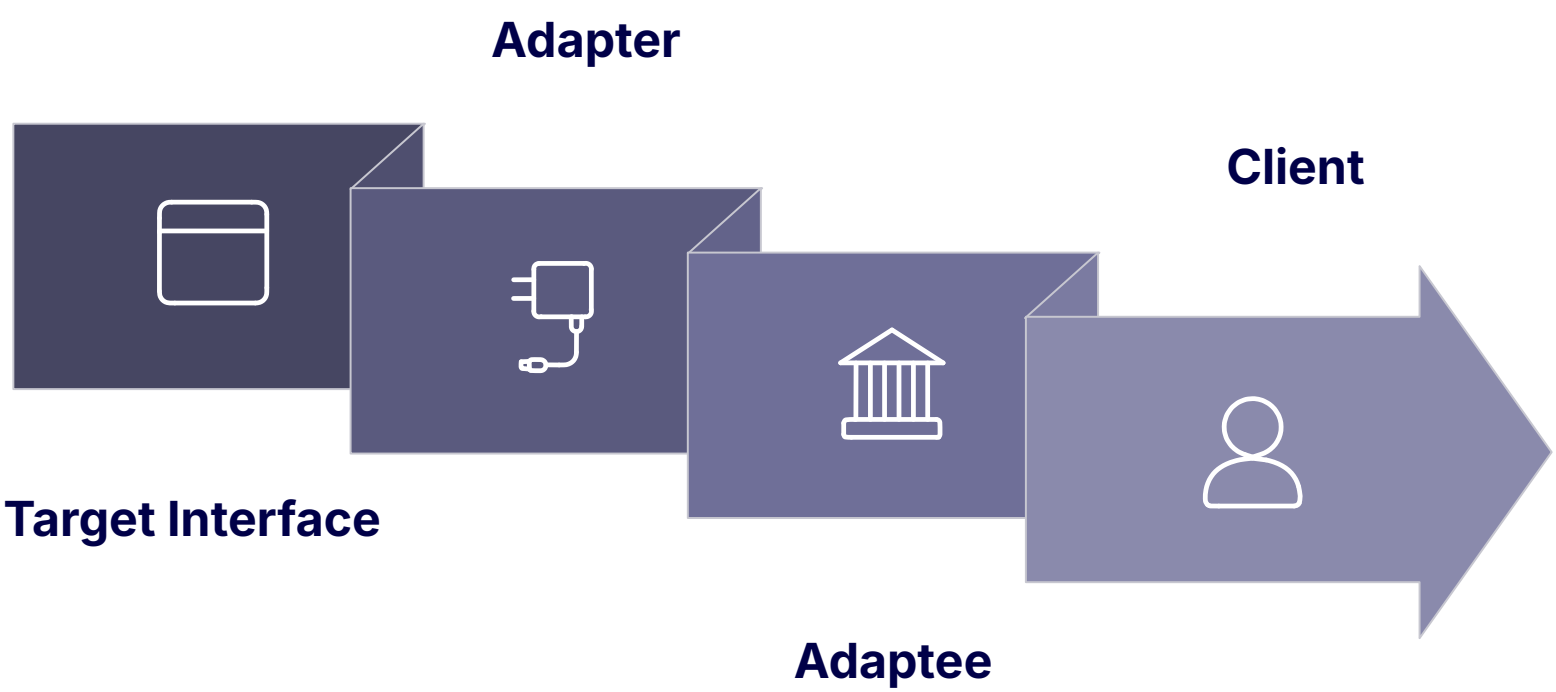
Reuse Old Code

Leverage existing implementations without rewriting them.

Improve Compatibility

Connect systems that were never designed to work together.

Adapter Pattern — In Practice



The Adapter sits between the client and the incompatible class, translating calls so both sides remain unchanged.

Target Interface

The interface the client expects to interact with.

Adaptee

The existing class with an incompatible interface.

Adapter

Wraps the Adaptee and implements the Target Interface.

Client

Interacts only with the Target Interface, unaware of the Adaptee.

Key discussion point: The Adapter pattern enables **wrapping existing functionality** and is essential in integration scenarios where you cannot modify the original source code.

Decorator Pattern

STRUCTURAL

Purpose: *Add functionality dynamically* to an object at runtime without altering its original class — wrapping objects in decorator layers to extend behaviour.

Real-World Examples

Coffee Customisation

Add milk, sugar, or syrup to a base coffee object without creating a new subclass for each combination.

Java InputStream Classes

`BufferedInputStream` wraps `FileInputStream` to add buffering — a classic Decorator in the JDK.

UI Component Enhancements

Add scrollbars, borders, or shadows to UI widgets dynamically.

Benefits

- **Flexible feature addition** — combine decorators in any order
- **Avoids subclass explosion** — no need for a class per feature combination
- **Single Responsibility** — each decorator handles one concern

Decorator Pattern — In Practice

Component Interface

Defines the common contract for both the base component and all decorators.

Concrete Component

The original object that provides the base behaviour to be extended.

Decorator Base Class

Holds a reference to a Component and delegates calls, allowing extension.

Concrete Decorators

Add specific behaviours by overriding methods and calling the wrapped component.

Composition over Inheritance: The Decorator pattern demonstrates how runtime behaviour extension through composition is more flexible than deep inheritance hierarchies.

Behavioural Patterns Overview

CATEGORY 3 OF 3

Goal

Improve **communication between objects**, defining how they interact and distribute responsibility effectively.

Patterns Covered

- Observer Pattern
- Strategy Pattern

Common Use Cases

Event Systems

Notify multiple listeners when an event occurs in the application.

Payment Methods

Switch between payment algorithms at runtime based on user choice.

Notification Systems

Broadcast updates to subscribers across different channels.

Observer Pattern

BEHAVIOURAL

Purpose: *Notify multiple objects automatically* when the state of a subject changes — establishing a one-to-many dependency without tight coupling.

Real-World Examples

YouTube Subscribers

All subscribers are notified when a channel uploads a new video.

Stock Market Apps

Multiple dashboards update simultaneously when a stock price changes.

Event Listeners

UI components react to user actions via registered event listeners.

Benefits

Loose Coupling

Subject and observers are independent — neither needs to know the other's details.

Easy Event Broadcasting

Add or remove observers at runtime without changing the subject.

Observer Pattern — In Practice

The publish-subscribe model at the heart of the Observer pattern powers event-driven architectures across modern Java applications.

Subject

Maintains a list of observers and notifies them on state change.

Observer Interface

Defines the `update()` method all concrete observers must implement.

Concrete Observers

Implement specific reactions to the subject's state changes.

The Observer pattern is the foundation of **event-driven architecture** — used extensively in Java's `java.util.EventListener` and reactive frameworks like RxJava.

Strategy Pattern

BEHAVIOURAL

Purpose: *Encapsulate interchangeable algorithms* behind a common interface, allowing the algorithm to vary independently from the clients that use it.

Real-World Examples

Payment Strategies

Switch between credit card, PayPal, or crypto payment at runtime.

Sorting Algorithms

Choose between QuickSort, MergeSort, or BubbleSort based on data size.

Navigation Routes

Select fastest, shortest, or scenic route strategies dynamically.

Benefits

- **Runtime behaviour switching** — change algorithms without modifying the context
- **Cleaner conditional logic** — eliminates complex `if-else` and `switch` chains
- **Open/Closed Principle** — add new strategies without touching existing code

Strategy Pattern — In Practice

1

Strategy Interface

Declares the common method (e.g. `execute()`) all strategies must implement.

2

Concrete Strategies

Each class encapsulates a specific algorithm variant (e.g. `CreditCardPayment`, `PayPalPayment`).

3

Context Class

Holds a reference to a Strategy and delegates execution — swappable at runtime.

The Strategy pattern is the cleanest way to **replace if-else chains** with polymorphism, fully honouring the **Open/Closed Principle** — your context never needs to change when a new strategy is added.

Design Patterns in Real Java Frameworks

Design patterns are not just academic concepts — they are **already embedded in the frameworks and libraries** Java developers use every day.

Framework / Library	Pattern Used	Where It Appears
Spring Framework	Singleton	Spring Beans are singletons by default in the application context
Java I/O	Decorator	`BufferedReader` wraps `FileReader` to add buffering
JDBC	Factory	`DriverManager.getConnection()` creates connections via factory logic
Java AWT / Swing	Observer	Event listeners implement the Observer pattern for UI events

Key Insight: Design patterns are already heavily used in enterprise Java applications. Recognising them in frameworks deepens your understanding and makes you a more effective Java developer.

Conclusion & Key Takeaways

Solve Recurring Problems

Design patterns provide battle-tested solutions to challenges every developer faces.

Improve Software Architecture

Patterns give your codebase a clear, intentional structure that scales with complexity.

Promote Clean Code

Well-applied patterns make code readable, maintainable, and a pleasure to work with.

Essential for Enterprise Java

Mastering patterns is a hallmark of senior Java developers working on production systems.

> "Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem." — **Gang of Four**

Thank you — Questions Welcome!